Анализ данных с использованием языка Python

Пояснения к коду. Семинар 4

Pandas: Series, DataFrame, загрузка и просмотр данных

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-04-pandas-basics.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 1–3, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/pandas.md`.

1 Зачем pandas после NumPy

Ячейка 1. Подключение pandas

```
1 # Импорт библиотеки pandas
2 import pandas as pd
```

– `import pandas as pd` — подключить pandas под коротким именем `pd`.

Ячейка 2. Загрузка таблицы

```
1
2 # Загрузка данных о ДТП в Брянской области из CSV-файла
3 dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id')  # ↩
  ↪ ДТП Брянской области
4 dtp.head()  # первые пять ДТП
```

– `pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id')` — прочитать таблицу из файла CSV. Путь задан от папки ноутбука: `..` — на уровень выше, из `notebooks` в папку репозитория, затем `datasets/legal`.

– `index_col='id'` — столбец `id`, номер ДТП, становится индексом: метками строк.

– `dtp.head()` — первые пять строк таблицы.

2 Series: столбец с метками

Ячейка 3. Series из списка

```
1 amounts = pd.Series([48_000, 250_000, 1_200_000, 75_000, 320_000, 15_000,
2                      640_000, 98_000, 410_000, 135_000, 27_000, 560_000],
3                     index=range(1, 13), name='amount')  # цены исков по ↩
  ↪ номерам дел
4 amounts
```

– `pd.Series([...], index=range(1, 13), name='amount')` — столбец из списка значений. `index=range(1, 13)` — метки от 1 до 12, номера дел; `name` — имя столбца, оно печатается внизу.

– Список записан на двух строках: пока скобка не закрыта, Python ждёт продолжения.

Ячейка 4. Значения и метки

```
amounts.values, amounts.index  # значения и метки
```

– `amounts.values` — значения: массив NumPy. `amounts.index` — метки; `RangeIndex` — метки подряд с шагом 1.

Ячейка 5. Арифметика

```
amounts / 1000  # цены в тысячах рублей
```

– `amounts / 1000` — деление каждого значения, как у массива; метки сохраняются.

Ячейка 6. Статистики

```
amounts.mean(), amounts.median(), amounts.max()  # статистики цен
```

– `mean()`, `median()`, `max()` — методы Series. Медиана у Series — метод, а у массива NumPy — функция `np.median`.

Ячейка 7. Метка наибольшего

```
amounts.idxmax()  # номер самого дорогого дела
```

– `amounts.idxmax()` — метка наибольшего значения, то есть номер дела. Позицию дал бы `argmax`.

Ячейка 8. Маска

```
amounts[amounts > 100_000]  # дела дороже 100 000
```

– `amounts[amounts > 100_000]` — отбор маской, как в NumPy; отобранные значения сохраняют свои метки.

Ячейка 9. По метке и по позиции

```
amounts.loc[3], amounts.iloc[3]  # дело 3 и четвёртое по счёту
```

– `amounts.loc[3]` — значение с меткой 3. `amounts.iloc[3]` — значение на позиции 3, четвёртое по счёту: позиции с нуля.

3 DataFrame и загрузка файла

Ячейка 10. Загрузка с явными параметрами

```
# Загрузка данных о ДТП в Брянской области из CSV-файла с явным указанием ↩
↪ параметров
dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id',
                  sep=',', encoding='utf-8')  # те же данные, параметры явно
dtp.shape  # число ДТП и столбцов
```

– `sep=','` — разделитель столбцов в файле; `encoding='utf-8'` — кодировка. Для файлов курса это значения по умолчанию; для файлов с российских порталов часто нужны `sep=';'` и `encoding='cp1251'`.

– `dtp.shape` — число строк и столбцов; индекс в число столбцов не входит.

Ячейка 11. Названия столбцов

```
dtp.columns  # названия столбцов
```

– `dtp.columns` — названия столбцов в порядке таблицы.

Ячейка 12. Типы столбцов

```
dtp.dtypes  # типы столбцов
```

– `dtp.dtypes` — тип каждого столбца: `str` — текст, `int64` — целые, `float64` — дробные.

Ячейка 13. Паспорт таблицы

```
dtp.info()  # строки, пропуски, типы
```

– `dtp.info()` — печатает число строк, диапазон индекса, для каждого столбца число заполненных значений (`Non-Null Count`) и тип, а внизу — память. Ничего не возвращает: только печатает.

Ячейка 14. Конец таблицы

```
dtp.tail(3)  # последние три ДТП
```

– `dtp.tail(3)` — последние три строки; без числа — пять.

Ячейка 15. Случайные строки

```
dtp.sample(3, random_state=4)  # три случайных ДТП
```

– `dtp.sample(3, random_state=4)` — три случайные строки. `random_state` — зерно: у всех одни и те же строки.

4 Первый взгляд на данные

Ячейка 16. Сводка числовых столбцов

```
dtp.describe().round(2)  # сводка числовых столбцов
```

– `dtp.describe()` — для каждого числового столбца число значений, среднее, стандартное отклонение, минимум, квартили 25%, 50%, 75% и максимум. Текстовые столбцы пропускаются.

– `.round(2)` — округлить все числа сводки до двух знаков.

Ячейка 17. Сводка одного столбца

```
dtp['participants_count'].describe()  # участники в одном ДТП
```

– `dtp['participants_count']` — один столбец по имени; `describe()` у Series даёт ту же сводку для одного столбца.

Ячейка 18. Частоты

```
dtp['category'].value_counts()  # ДТП по видам
```

– `value_counts()` — сколько раз встречается каждое значение столбца, по убыванию. Результат — Series: значения столбца стали метками, числа — значениями.

Ячейка 19. Доли

```
dtp['severity'].value_counts(normalize=True).round(3)  # доли по тяжести
```

– `value_counts(normalize=True)` — доли вместо чисел: каждое число делится на число заполненных строк. Пропуски в доли не входят.

Ячейка 20. Сколько разных значений

```
dtp['light'].nunique(), dtp['light'].unique()  # условия освещения
```

– `nunique()` — число разных значений; `unique()` — сами значения в порядке первого появления.

Ячейка 21. Среднее по тексту: намеренная ошибка

```
dtp['category'].mean()  # среднее по тексту: ошибка
```

– `mean()` по текстовому столбцу: `TypeError`, статистику по строкам не посчитать.

5 Столбцы и строки

Ячейка 22. Один столбец

```
dtp['dead_count']  # столбец погибших: Series
```

– `dtp['dead_count']` — столбец по имени в квадратных скобках: Series с индексом таблицы. Длинную Series pandas показывает началом и концом, через две точки.

Ячейка 23. Несколько столбцов

```
dtp[['datetime', 'district', 'dead_count']].head()  # три столбца: таблица
```

– `dtp[['datetime', 'district', 'dead_count']]` — внешние скобки — выбор из таблицы, внутренние — список имён. Результат — таблица из этих столбцов в заданном порядке.

Ячейка 24. Без внутренних скобок: намеренная ошибка

```
dtp['datetime', 'district']  # без внутренних скобок: ошибка
```

– `dtp['datetime', 'district']` — два имени через запятую без списка pandas понимает как одно составное имя `('datetime', 'district')`; такого столбца нет, поэтому `KeyError`.

Ячейка 25. Строка по метке

```
dtp.loc[31]  # ДТП номер 31
```

– `dtp.loc[31]` — строка с меткой 31, ДТП номер 31. Результат — Series: индекс — названия столбцов, значения — поля этого ДТП.

Ячейка 26. Одно поле

```
dtp.loc[31, 'district']  # район этого ДТП
```

– `dtp.loc[31, 'district']` — до запятой метка строки, после — имя столбца: одно значение.

Ячейка 27. Строка по позиции

```
dtp.iloc[2]  # третье ДТП по счёту
```

– `dtp.iloc[2]` — строка на позиции 2, третья по счёту. Её метка, номер ДТП, видна внизу вывода в строке `Name`.

Ячейка 28. Несколько строк и столбцов

```
dtp.loc[[31, 133], ['datetime', 'category', 'dead_count']]  # два ДТП, три ↩
↪ поля
```

– `dtp.loc[[31, 133], [...]]` — список меток строк и список имён столбцов: таблица на пересечении.

Ячейка 29. Срез по позициям

```
dtp.iloc[:3]  # первые три ДТП
```

– `dtp.iloc[:3]` — строки на позициях 0, 1 и 2: правая граница не входит.

Ячейка 30. Срез по меткам и по позициям

```
amounts.loc[3:5], amounts.iloc[3:5]  # метки включительно, позиции нет
```

– `amounts.loc[3:5]` — метки от 3 до 5 включительно: в срезе `loc` правая граница входит.

– `amounts.iloc[3:5]` — позиции 3 и 4: в срезе `iloc` правая граница не входит.

6 Маски

Ячейка 31. Маска ДТП с погибшими

```
with_dead = dtp['dead_count'] > 0  # маска: ДТП с погибшими
with_dead.sum(), with_dead.mean().round(3)  # число и доля
```

– `dtp['dead_count'] > 0` — маска: Series из `True` и `False` с индексом таблицы.

– `with_dead.sum()` — число ДТП с погибшими; `with_dead.mean().round(3)` — их доля, три знака.

Ячейка 32. Отбор строк

```
dtp[with_dead].head()  # сами ДТП с погибшими
```

– `dtp[with_dead]` — маска в квадратных скобках таблицы оставляет строки, где `True`; `head()` — первые пять из них.

Ячейка 33. Вхождение в список

```
night = dtp['light'].isin(['В темное время суток, освещение включено',
                           'В темное время суток, освещение отсутствует',
                           'В темное время суток, освещение не включено']) # маска: ночные ДТП
night.sum(), night.mean().round(3)  # число и доля ночных
```

– `dtp['light'].isin([...])` — маска: `True`, если значение есть в списке. Здесь список из трёх значений тёмного времени, записанный на трёх строках.

Ячейка 34. Два условия

```
pedestrians_night = (dtp['category'] == 'Наезд на пешехода') & night  # пешеходы ночью
pedestrians_night.sum()
```

– `(dtp['category'] == 'Наезд на пешехода') & night` — «и» двух масок; первое условие в скобках, вторая маска уже готова.

Ячейка 35. В интервале

```
dtp['injured_count'].between(3, 5).sum()  # ДТП с 3-5 ранеными
```

– `between(3, 5)` — маска: значение от 3 до 5, обе границы включены. `sum()` — сколько таких ДТП.

Ячейка 36. and вместо &: намеренная ошибка

```
(dtp['category'] == 'Наезд на пешехода') and night  # and вместо &: ошибка
```

– `and` спрашивает у маски одно «да» или «нет»; у маски их тысячи, отсюда `ValueError`. Для масок — только амперсанд.

Ячейка 37. Маска и столбцы в loc

```
dtp.loc[dtp['dead_count'] >= 3, ['datetime', 'district', 'category', 'dead_count']]  # ДТП с тремя и более погибшими
```

– `dtp.loc[маска, [...]]` — до запятой маска отбирает строки, после — список столбцов: выборка нужных полей.

7 Мини-анализ и сохранение

Ячейка 38. Доля наездов на пешеходов

```
(dtp['category'] == 'Наезд на пешехода').mean().round(3)  # доля наездов на пешеходов
```

– Сравнение, его среднее и округление в одной цепочке: `(...).mean().round(3)`.

Ячейка 39. Днём и в тёмное время

```
day = dtp['light'] == 'Светлое время суток'  # маска: дневные ДТП
(dtp.loc[day, 'dead_count'] > 0).mean().round(3), (dtp.loc[night, 'dead_count'] > 0).mean().round(3)  # доля ДТП с погибшими днём и ночью
```

– `dtp.loc[day, 'dead_count']` — столбец погибших только у дневных ДТП; `> 0` — маска ДТП с погибшими среди них; `mean()` — их доля.

– Вторая половина строки — то же для маски `night` из ячейки 33.

Ячейка 40. Самое тяжёлое ДТП

```
worst = dtp['dead_count'].idxmax()  # номер самого тяжёлого ДТП
dtp.loc[worst]
```

– `dtp['dead_count'].idxmax()` — метка, номер ДТП с наибольшим числом погибших. `dtp.loc[worst]` — его карточка. `NaN` в карточке — пропуск: недостатки дороги не установлены.

Ячейка 41. Сохранение в файл

```
severe = dtp[dtp['dead_count'] >= 3]  # ДТП с тремя и более погибшими
severe.to_csv('dtp-bryansk-severe.csv')  # выборка в файл
len(severe)
```

– `severe = dtp[...]` — выборка ДТП с тремя и более погибшими.

– `severe.to_csv('dtp-bryansk-severe.csv')` — записать таблицу в файл CSV рядом с ноутбуком; индекс записывается первым столбцом.

– `len(severe)` — число строк выборки.